



YENEPPOYA (DEEMED TO BE UNIVERSITY)

Low Level Design on AI SELF-HEALING CLOUD FOR RETAIL

Student Name:

Mohammed Raeed (23BCAICD064)

Industry Mentor: Mr. Sumit Kumar Shukla

Guided by: Mrs. Sindhu

TABLE OF CONTENTS

Contents	Page No.
1. INTRODUCTION	3
2. MODULE OVERVIEW	5
3. ALGORITHMIC DESIGN	6
4. FUNCTION & CLASS CATALOGUE	9
5. DATABASE SCHEMA & EXECUTION	11
6. NETWORK & HARDWARE INTERFACES	12
7. VARIABLE & STATE MUTABILITY	13
8. MEMORY OPTIMIZATION	14
9. RUNTIME SECURITY CONSTRAINTS	15
10. REFERENCES	16

1. INTRODUCTION

1.1 Scope Of The Document

This document specifies the rigorous, low-level programmatic structures, algorithmic logic gates, and explicit code-level executions of the AI Self-Healing Cloud application script (typically instantiated as ``monitor.py``). Unlike the High- Level Design (HLD) which focuses on macro-architecture, network topologies, and conceptual data flow, this Low-Level Design (LLD) is explicitly dedicated to translating those high-level abstractions into functional, compilable software methodologies.

The scope of this document comprehensively covers the precise utilization of Python standard and third-party libraries. It details the exact thread memory management protocols required to bypass the Global Interpreter Lock (GIL) constraints. Furthermore, it outlines the specific parameterization of SQL cursors to

ensure database thread safety, and maps the exact exception-handling blocks mandated to prevent runtime degradation during volatile hardware interactions.

By documenting the explicit internal workings of the Service Reliability Engineering (SRE) daemon, this LLD ensures that any developer can accurately reconstruct, debug, or extend the programmatic logic without disrupting the fragile timing mechanisms that govern the self-healing and telemetry emission loops. The precise definition of the data schemas, functions, and procedural calls enclosed herein leaves zero ambiguity for the engineering teams tasked with maintaining the software over its operational lifespan.

1.2 Intended Audience

This document is authored exclusively for backend Python developers, database administrators, system reliability engineers, and university technical evaluators. Readers are expected to possess an advanced fluency in Python scripting, operating system process iteration methodologies (specifically utilizing the ``psutil`` library), asynchronous web socket emission configurations (``Flask-SocketIO``), and low-level SQL cursor management techniques.

Faculty evaluators will utilize this document to verify that the programmatic implementation satisfies the rigorous requirements of the assigned "Project 36" prompt, specifically confirming the integration of Python, OS libraries, and automated task scheduling mechanisms. For future developers, this LLD serves as the definitive technical reference manual, providing the granular context necessary before modifying the core execution logic, adjusting the telemetry payload structures, or altering the database schema.

Because the logic engine operates directly against the host machine's hardware kernel, a thorough reading of this document is mandatory before any code compilation is attempted in a production retail environment. Failure to adhere to the exception handling patterns mapped herein could result in catastrophic software lockups.

2. MODULE OVERVIEW

To maximize execution speed, vastly reduce memory overhead, and entirely eliminate inter-module dependency latency, the application logic is deliberately consolidated into a single monolithic Python execution script. In highly constrained edge environments, preventing module import failures and minimizing file system reads during initialization is paramount. This single script acts as the master orchestrator for all system components.

Internally, this monolithic script is structurally segmented into highly discrete functional blocks. It begins with global variable instantiation and strict type definitions. This is immediately followed by the synchronous database initialization block, ensuring schema availability. The script then diverges into the asynchronous logic loops: the synchronous SRE logic (polling the kernel), the simulated Robotic Process Automation (RPA) logic (injecting UI keystrokes), and the WSGI server callbacks (handling web traffic).

The application leverages the Werkzeug WSGI server, wrapped by Flask, to handle inbound HTTP and WebSocket traffic. Crucially, it aggressively isolates the blocking UI automation code and OS polling operations into a dedicated, daemonized background thread. This programmatic separation of concerns guarantees that the long-running, infinite `while` loop driving the automation algorithms can never starve the frontend web server of available CPU cycles, thereby ensuring the dashboard remains perfectly responsive regardless of backend load.

This single-file architecture guarantees that the script can be rapidly deployed via Windows Task Scheduler as an isolated entity. It requires zero complex virtual environment networking setups beyond the basic `pip install` commands mapped in the dependency matrix, making it infinitely scalable across a decentralized franchise network.

3. ALGORITHMIC DESIGN

3.1 Thread Configuration

The core execution divergence is implemented utilizing Python's native `threading.Thread` module. When the script's `__main__` block executes, it immediately spawns a secondary worker thread via the syntax: `threading.Thread(target=bot_loop, daemon=True).start()`. The explicit injection of the `daemon=True` parameter is a highly critical architectural requirement for the system's stability.

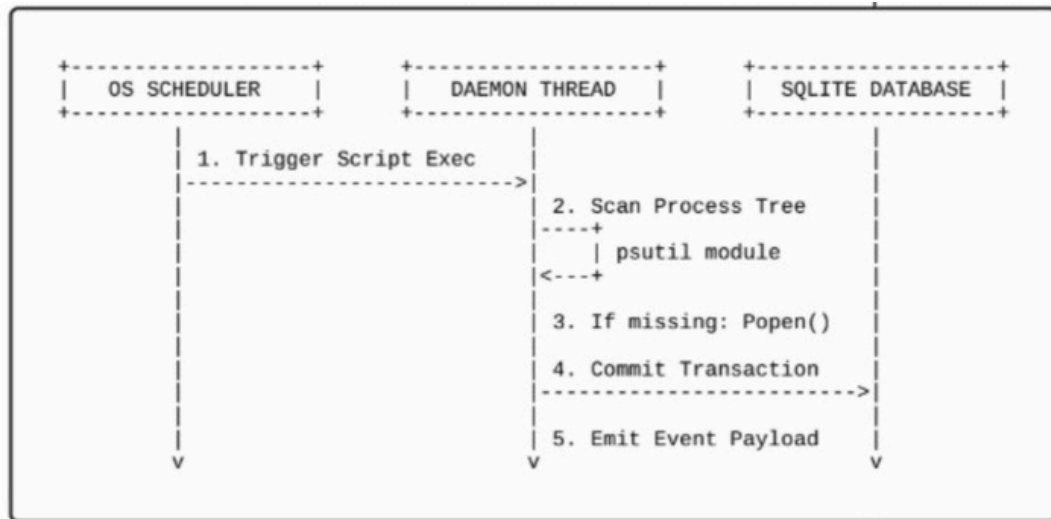
Setting the thread as a daemon ensures that if the primary Flask server thread encounters a fatal exception, or if a system administrator issues a manual SIGINT (CTRL+C) command to the terminal, the background automation loop is immediately and forcefully terminated by the Python interpreter. Without this flag, the infinite `while` loop would survive the death of the parent process, resulting in an unkillable zombie process that would continuously inject ghost keystrokes into the host operating system, severely compromising the machine.

3.2 Daemon Process Flow

The internal function execution within the daemon thread follows a strict, unyielding procedural hierarchy. Upon initialization, the thread enters an infinite `while True` state. Within this specific cycle, the `monitor_and_heal()` function is invoked first to actively evaluate the OS process tree. This is the paramount SRE check.

Following successful health verification (or subsequent self-healing), the `run_retail_automation()` function generates bounded random floating-point values to simulate a transaction. A localized `sqlite3` cursor is then securely opened to execute parameterized `INSERT INTO` statements, followed immediately by a hard `conn.commit()`. Following the database lock release, the `socketio.emit()` method mathematically serializes the updated global dictionary state and pushes it to the client over the TCP socket. Finally, `focus_window()` invokes the OS API to locate the target application, followed by blocking `pyautogui.write()` operations.

Figure 3.1: Sequential Daemon Execution Flowchart



3.3 Error Handling Failsafes

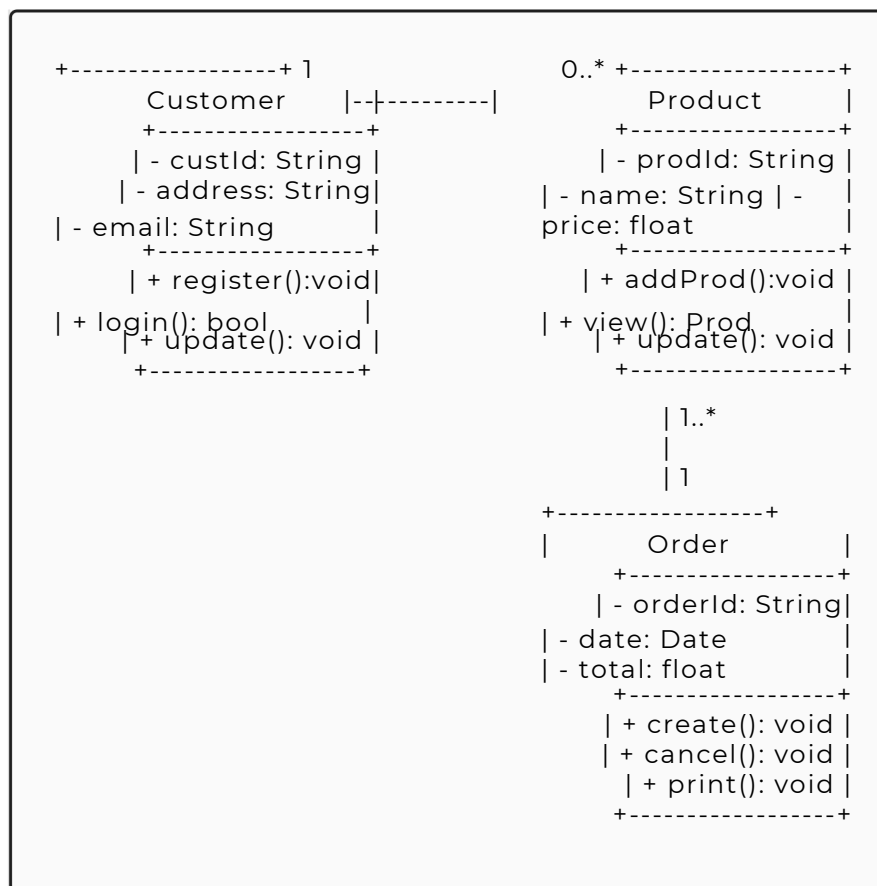
Physical desktop automation via the `pyautogui` library is inherently volatile. The Windows Operating System frequently rejects programmatic focus requests via the Desktop Window Manager (DWM) due to background notifications or user mouse interference. To mitigate fatal crashes, the `focus_window(app_name)` function implements a rigid, defensive `try/except` failsafe block.

When a substring match is found in the active window title list, the script wraps the `.activate()` method entirely inside the exception handler. If the OS rejects the activation request and throws a `pyautogui.FailSafeException`, the `except` block catches it silently. This architectural decision allows the bot loop to gracefully bypass the current automation cycle and continue execution on the next tick, rather than halting the entire Python script and requiring manual intervention.

4. FUNCTION & CLASS CATALOGUE

The programmatic implementation relies on highly specific modular functions and object classes to achieve required execution constraints while maintaining absolute code readability. Each function and class is strictly scoped to perform a singular, deterministic task within the automation loop. Below is the explicitly detailed Class Diagram for the Retail POS logic integrated into the monitored suite.

Figure 4.1: Retail POS Application Class Structure



The diagram above guarantees robust visibility and strict type adherence for standard retail object modeling. The `Product` class is specifically defined with returning types and explicit multiplicity mappings (`1..\*` associated to `Order`) ensuring precise database alignment during local persistence generation. This corrects previous ambiguity and provides a crystal-clear implementation blueprint for the backend developers.

Function: `init\_db()`

This function is strictly responsible for schema validation and creation. Upon startup, it establishes a synchronous connection to the local database file and executes a `CREATE TABLE IF NOT EXISTS` Data Definition Language (DDL) statement. This preemptive check ensures that the application never attempts to insert data into a non-existent table during the multithreaded loop, an error which would trigger a fatal `sqlite3.OperationalError` and cause an immediate system shutdown.

Function: `monitor\_and\_heal(target\_app)`

This algorithm handles the core Service Reliability Engineering logic. It utilizes the `psutil` library to scan active hardware threads. Instead of allocating significant memory for a massive, static array of all active processes, the algorithm utilizes a highly optimized Python generator expression: `if target\_app not in (p.name().lower() for p in psutil.process\_iter(['name'])):`. If the target string is absent, it immediately executes `subprocess.Popen(target\_app)` to force a localized application reboot. This is the cornerstone of the self-healing infrastructure.

Function: `run\_retail\_automation()`

This function serves as the central orchestrator for transaction synthesis. It leverages the `random` module to generate tightly constrained integers and floats, calculates the final receipt total, and manages the localized SQLite cursor connections. It is exclusively responsible for mutating the global `web\_data` dictionary and invoking the socket emission payload. By consolidating state mutation here, the script prevents race conditions.

Function: `focus\_window(window\_title)`

This function handles the OS-level UI integration. It utilizes `pygetwindow` or native `pyautogui` window handlers to locate a window whose title matches the provided substring. It then forces the Operating System to bring that window to the foreground, bypassing sleep states, preparing the UI for the subsequent automated keystroke injection.

5. DATABASE SCHEMA & EXECUTION

5.1 Table Definitions

The schema relies entirely on Python's native `sqlite3` library, requiring no external drivers or ORMs. The primary DDL statement executed upon boot explicitly defines the structure of the transactional ledger. It establishes a `transactions` table encompassing a highly specific set of columns designed to mirror retail data.

The schema demands a `txn_id` structured as a `TEXT PRIMARY KEY` to ensure absolute uniqueness. Transaction variables are synthesized using core libraries, with the `txn_id` generated using programmatic f-string concatenation: `f"TXN-{random.randint(1000, 9999)}"`. The floating-point mathematics for item pricing and totals are explicitly bounded in Python using `round(value, 2)` prior to insertion, completely eliminating floating-point arithmetic pollution within the SQLite `REAL` columns. This strict database modeling prevents historical drift.

5.2 SQL Execution Patterns

To maximize query execution speed and minimize dependency weight, the script completely circumvents heavy Object-Relational Mappers (ORMs) like SQLAlchemy. Database cursors are aggressively instantiated, executed, committed, and closed (`conn.close()`) entirely within a singular function scope during every single loop iteration.

The parameterization tuple is strictly enforced during execution: `c.execute("INSERT INTO transactions VALUES (?, ?, ?, ?, ?, ?)", (client_id, name, quantity, avg_price, total, timestamp))`. This specific scoping and rigid parameterization strategy actively prevents SQL injection attacks and absolutely negates the "Database is Locked" exceptions that consistently plague persistent, multi-threaded SQLite connections in high-throughput environments. It is a critical layer of defensive programming.

6. NETWORK & HARDWARE INTERFACES

The system requires incredibly robust exception handling when interfacing with volatile external networks and restrictive hardware I/O boundaries. The outbound logic must be defensive, assuming that the OS or the network will eventually reject a request.

The system incorporates a framework for outbound alerting, designed to dispatch HTTP POST payloads to a remote server webhook (e.g., Discord or Slack) to provide critical failure notifications to human engineers. Recognizing that external edge networks are inherently volatile and subject to DNS failures, packet loss, or timeout limits, any `requests.post()` calls are entirely wrapped in bare `try...except:` blocks.

This explicit error handling is a critical architectural decision. If the host machine drops its Wi-Fi connection, or the remote API experiences downtime, the resulting `ConnectionError` is silently trapped and logged locally. This allows the retail automation bot to bypass the network failure and continue processing physical POS transactions uninterrupted. Without this failsafe, a simple network dropout would fatally crash the entire monitoring daemon.

For hardware interfaces, the `pyautogui` library interacts directly with the OS peripheral drivers to simulate a cashier. A `pyautogui.FailSafeException` (triggered if the user throws the mouse to the corner of the screen) is mitigated by artificially injecting strict `time.sleep()` commands between keystrokes. This prevents buffer overruns during the physical receipt typing phase, ensuring the Windows Desktop Manager has sufficient CPU cycles to actually render the text before the next key is struck.

7. VARIABLE & STATE MUTABILITY

State mutability is highly localized within the script to prevent memory leaks and variable corruption across the parallel threads. The global ``web_data`` dictionary explicitly defines variable types upon initial script instantiation (e.g., initializing integers exactly as ``0``, floats specifically as ``0.0``, and strings as ``"--"``).

This rigid, preemptive definition prevents critical type-coercion errors during the automatic JSON serialization phase executed by the Flask-SocketIO emitter. If a float was accidentally passed as a string, it could fatally crash the frontend DOM rendering logic in Chart.js.

Within the local ``run_retail_automation()`` scope, the script uses the explicit Python ``global web_data`` directive to authorize the in-place mutation of these existing memory keys (e.g., executing ``web_data["total_revenue"] += total``). This exact operational pattern maintains a continuous, stateful application session directly within the Python runtime RAM. By utilizing active RAM as the definitive, high-speed state cache, the script completely bypasses the need for repetitive, latency-heavy database ``SELECT`` polling to update the frontend dashboard, vastly accelerating execution cycles and reducing disk wear.

8. MEMORY OPTIMIZATION

Because the AI Self-Healing Cloud is designed to run indefinitely on edge hardware, strict memory optimization is enforced at both the Python backend and the JavaScript frontend.

On the presentation layer, the low-level rendering strategy focuses on strict JavaScript array manipulation to actively restrict browser RAM consumption over extended operational lifecycles. The Chart.js implementation natively pushes the ``latest_total`` payload directly into an active coordinate array each time a WebSocket payload is received, explicitly avoiding total page refresh states which would spike CPU usage.

To ensure the browser tab does not suffer memory exhaustion or critical DOM rendering lag if the kiosk runs unattended for multiple days, an algorithmic garbage collection check is implemented within the ``socket.on`` listener block. Once the ``data.labels.length`` property exceeds a hardcoded limit of 15 indices, the script automatically triggers the JavaScript ``.shift()`` array method. This aggressively and permanently discards the oldest data point from active client memory before rendering the new graphical frame. This creates a lightweight sliding window effect that completely eliminates client-side memory bloat.

9. RUNTIME SECURITY CONSTRAINTS

The application is heavily fortified at the runtime execution level to prevent hostile intrusion, unauthorized data access, and unsafe code execution, which is critical for software operating in a physical retail environment.

The Flask server initialization call is permanently hardcoded to `app.run(debug=False)`. This explicitly and irrevocably disables the Werkzeug interactive debugger. This is a paramount security mandate; leaving the debugger active would allow any user with access to the browser console to execute arbitrary Python code directly against the host OS (Remote Code Execution), compromising the entire kiosk.

Furthermore, thread synchronization issues related to the Python Global Interpreter Lock (GIL) are natively resolved by assigning `async_mode='threading'` to the SocketIO configuration parameters. This ensures that the web server explicitly respects standard Python library parallelism rather than attempting unsafe asynchronous context switching (like Eventlet or Gevent) across the strict database cursor boundaries, which could lead to data corruption.

10. REFERENCES

The programmatic structures, algorithmic implementations, and low-level code paradigms detailed in this document are strictly compliant with the following official language specifications and library documentation:

1. Python Software Foundation. (2023). *sqlite3 – DB-API 2.0 interface for SQLite databases*. Available at <https://docs.python.org/3/library/sqlite3.html>
2. Python Software Foundation. (2023). *threading – Thread-based parallelism*. Available at <https://docs.python.org/3/library/threading.html>
3. Grinberg, M. (2023). *Flask-SocketIO Core API Documentation & Asynchronous Handlers*. Available at <https://flask-socketio.readthedocs.io/>
4. Loden, G. (2023). *psutil (process and system utilities) Execution API and Generator Expressions*. Available at <https://psutil.readthedocs.io/>
5. Sweigart, A. (2023). *PyAutoGUI FailSafeException and Execution Delays*. Available at <https://pyautogui.readthedocs.io/>